面试复盘：从「课程项目」到「有评测的 Agent 项目」

> 用途：面试前 30 分钟过一遍；任何一条被问到都能答到点子上。 数据截止：2026-11-13，版本 `v1.4-nine-metrics`。

一、你改了什么（对比"刚加评测之前"）

1. 时间线（每个阶段：改了什么 → 为什么 → 数据）

阶段	改动	为什么	结果（实测）
起点	课程项目：RAG + 7 工具 + `create_agent`，工具全是 mock（`random`），无评测	—	知识库 37 块；无任何指标
① 知识库	`扫地机器人100问.txt` 从 10 问补到 **100 问 / 11 类**	原内容太少，RAG 无法覆盖真实问题	块数 37 → **88**
② 外部数据	`records.csv` 28 → **68 条**（补 2026-08~11）	报告功能"查上个月"永远查不到数据	10 用户 × 7 个月
③ 配置化	`agent.yml` 加 `mock_user`；`USER_ID/USER_CITY` 支持环境变量；`get_current_month` 改真实时间	mock 随机导致评测不可复现（同一用例两次结果不同）	评测可复现；T001 由 FAIL → PASS
④ 路径修复	`persist_directory` 走 `get_abs_path()`；统一到 `rag/chroma_db`；删空库	曾生成两份向量库（`agent/chroma_db` 空库）	只剩 1 个库
⑤ 提示词	从"信息不足才调工具"改为**七条硬约束**（强制检索/禁编造/回答完整/先追问/禁补充/保密/超范围拒答）；修复报告约束段乱码	模型凭常识跳过检索、编造数值、不追问、弱拒答	检索类 66.7%→86.7%；追问 0%→100%；拒答 →100%
⑥ 工具	`get_weather` 接真实 `wttr.in`（超时 + 兜底返回空串）；`fetch_external_data` 加**越权拦截** + 返回 JSON	全 mock 无法体现工程能力；越权返回他人数据	参数准确率 100%；安全 50%→100%
⑦ 评测体系	从 0 建到 **30 条用例 + 九类指标 + 判据版本化 + 预检/重试/基线元数据**	没有尺子，所有改动都不可证	见下表
⑧ 文档与仓库	README、评测报告（§0–§11）、4 份 Release 说明、待开 Issue、2 份 PDF 讲义、思维导图、5 个 tag	让面试官能"看得到过程"	12+ commit / 5 tag / 20 份跑批明细

2. 评测体系的演进（这张表最值钱）

版本	用例	判据	模型	通过率	关键动作
v1.0	25	j1	qwen-max	72.0%	首个可复现基线
v1.1	26	j2	qwen-max	73.1%	加报告用例 T005、mock 配置化
v1.2-a	26	j3	qwen-max	**88.5%**	P0 提示词（同模型同判据，**唯一干净对比**）
v1.2-b	26	j3	qwen-plus	**96.2%**	判据修正（破折号/同义词）
v1.3	30	j4	qwen-plus	86.7%	六类指标 + 4 条安全用例
v1.4	30	j5	qwen-plus	86.7%	九类指标 + P0（忠实度 40→70%、安全 50→100%、拒答 100%）

3. 最终指标（v1.4，30 条）

层	指标	数值
生成	通过率	86.7%
生成	忠实度（judge）	70.0%（原 40%）
生成	答案相关性	均分 3.07，46.7% ≥3.5
检索	hit@3	93.8%
检索	src@1 / @3 / @5	81.2% / 100% / 100%
检索	MRR@5（文件级）	0.896
工具	调用 / 参数准确率	95.2% / 100%
行为	拒答 / 安全	100% / 100%
工程	tokens（入/出）	5010 / 250
工程	单次成本	0.00432 元
工程	延迟	P50 约 12–16s（见报告）

4. 代码级清单（谁改了什么）

文件	改动
`prompts/main_prompt.txt`	五条→**七条硬约束**；报告流程 5 步化；修乱码
`agent/tools/agent_tools.py`	mock→配置注入；`get_weather` 真实 HTTP；越权拦截；JSON 返回；`CURRENT_USER_ID`
`agent/tools/middleware.py`	工具监控 / 模型前日志 / `@dynamic_prompt` 报告提示词切换
`agent/react_agent.py`	`create_agent` + 7 工具 + 3 中间件；`execute_stream`
`model/factory.py`	工厂；`max_tokens=2000` 防截断
`rag/vector_store.py`	`persist_directory` 绝对化
`config/agent.yml` · `rag.yml` · `chroma.yml`	用户配置、模型切换、库路径
`data/扫地机器人100问.txt`	10 → **100 问**（11 类）
`data/external/records.csv`	28 → **68 条**
`eval/run_eval.py`	评测引擎 v4：九指标、judge、预检、重试、基线元数据
`eval/regrade.py`	**离线重打分**（额度不足时用）
`eval/prices.yml`	成本核算价格表
`eval/dataset.jsonl`	30 条用例（6 类）+ `gold_source/expect_args/must_not_contain`
`docs/`	评测报告（§0–§11）、Release ×4、待开 Issue

二、你必须掌握的东西（按"被问到的概率"排序）

A. 必须能张口就讲（不讲就露馅）

1. **整条链路**：用户问题 → agent 判断 → 调 `rag_summarize` → 检索 top-3 → 拼 context → 提示词 → 模型 → 答案；
2. **九类指标各自定义与公式**（尤其 hit@3、MRR@5、忠实度）；
3. **为什么 96.2% 和 86.7% 不能比**（模型/判据/数据集三不可比）；
4. **判据 bug 的真实例子**：`hit@3` 不支持同义词导致 81.2%；破折号 `2–3厘米` 被判失败；
5. **忠实度 40% 的根因与修复**（模型补充资料未提及的数值 → 提示词第 5 条）；
6. **越权怎么防**（`user_id == CURRENT_USER_ID` 校验）；
7. **为什么换 qwen-plus**（qwen-max 免费额度耗尽；基线按模型分开存）。

B. 应该能解释（追问两层以内）

1. `chunk_size=200 / overlap=20 / k=3` 的取舍，以及下一步怎么调；
2. `RecursiveCharacterTextSplitter` 为什么"递归"、`split_documents` 与 `split_text` 的区别（元数据）；
3. `@dynamic_prompt` + `fill_context_for_report` 的"信号灯"机制；
4. 中间件三类钩子的执行时机（`wrap_tool_call` / `before_model` / `dynamic_prompt`）；
5. judge 与规则评分的分工，以及 judge 为什么必须校准；
6. 成本公式（tokens × 单价）与"输入/输出必须分开统计"的原因；
7. 为什么评测要用 `--no-llm` / `--limit` / `regrade.py`（省钱与快速验证）。

C. 加分项（能说出来很亮）

1. 判据版本号（j1→j5）与"跨版本不可比"的纪律；
2. 基线元数据（model / dataset_sha256 / git_commit）保证可复现；
3. 预检 + 重试退避 + 防空基线（企业级评测的三道保险）；
4. 离线重打分（额度不足时仍能验证判据）；
5. 用"信号灯工具"实现同一 agent 双模式（客服/报告）切换。

三、面试可能追问的 22 个问题 + 正确回答

1. 用一句话介绍这个项目?

> "一个扫地机器人智能客服 Agent：LangChain 1.x `create_agent` + 7 个工具 + Chroma 知识库（88 块），核心是我自建的一套评测体系——30 条用例、九类指标、可复现基线，用它把忠实度从 40% 拉到 70%、安全从 50% 拉到 100%。"

2. 为什么用 RAG 而不是微调?

> "知识是产品文档、更新频繁，微调成本高且难以追溯依据；RAG 能把答案锚定到具体文件（我保留了 `source` 元数据），改文档即生效；而且我没有标注数据，微调不现实。"

3. `chunk_size=200`、`k=3` 是怎么定的?

> "课程给的保守起点。200 字大约对应一条问答，重叠 20 字防止句子被切断；k=3 是精度/召回折中。下一步我会用同一套评测把 k 提到 5、chunk 提到 400，用 hit@3 和 MRR@5 对比——**参数不是我拍脑袋，是要用评测说话。**"

4. hit@3 和 MRR@5 有什么区别?

> "hit@3 是二值：前 3 块能否覆盖全部关键事实；MRR@5 看**排在第几**——第一个命中 `gold_source` 的位置取倒数再平均，现在是 0.896。前者告诉我'够不够'，后者告诉我'排序好不好'。注意我的 MRR 是**文件级**，因为没有 chunk 级标注，这点我会主动说明。"

5. 忠实度怎么测? judge 可信吗?

> "我用 LLM-as-Judge：把问题、top-3 参考资料、模型回答一起给 judge，让它判断关键事实是否都能在资料中找到，并列出'无依据的点'。**judge 现在还没校准**——计划人工标 20 条算一致率（目标 ≥80%），所以目前 70% 只作相对趋势，不作为对外结论。"

6. 你的通过率从 96.2% 掉到 86.7%，是退步吗?

> "不是。96.2% 是 26 条用例、旧判据、qwen-plus；86.7% 是 30 条（多了 4 条安全用例）、新判据 j5。**三个变量同时变了，不可横比。**真正可比的只有 v1.2-a：同模型同判据同数据，73.1%→88.5%。我把这条纪律写进了 README 和评测报告 §0。"

7. 判据也会错? 举个例子。

> "会。第一版 `hit@3` 不支持'同义词或'写法，导致 81.2% 的假失败；qwen-plus 输出 `2–3厘米`（en dash），我的归一化没处理，判成事实错误；还有拒答，模型说'这不在我的服务范围'，我的词表只有'不在服务范围'，漏判了 3 条。**判据比模型更影响结论**，所以我引入了判据版本号。"

8. 怎么保证评测可复现?

> "四件事：① 温度默认、固定数据集；② baseline 里记录 `model / dataset_sha256 / git_commit / judge_version`；③ 脚本发现模型或判据版本不同就拒绝横比；④ 每轮跑批的原始记录（工具调用、答案、耗时、tokens）落盘到 `eval/results/`，可回放。"

9. 为什么换模型？换了怎么保证公平？

> "qwen-max 免费额度耗尽（403 `AllocationQuota.FreeTierOnly`）。换 qwen-plus 后我给基线加了模型字段，**按模型分开存基线**（`baseline.json` 是 qwen-max，`baseline_qwen_plus*.json` 是 qwen-plus），脚本比对时发现模型不同直接拒绝横比。"

10. 工具参数准确率怎么算？为什么单独统计？

> "对带 `expect_args` 的用例，检查模型实际 `tool_calls` 里的参数：`user_id=1001`、`month=2024-02` 精确匹配，`city` 只要求存在（用 `*`），月份用正则 `^\d{4}-\d{2}$`。单独统计是因为**只查'调没调'发现不了'传错参'**——我曾把'上个月'算成 2023-12，就是参数/推理错。"

11. 越权是怎么防的？够不够？

> "在 `fetch_external_data` 入口校验 `user_id == CURRENT_USER_ID`，不一致直接返回'无权查询其他用户'，并打 warning 日志。**这只能挡住这一条路径**：真实系统应该在网关/会话层做身份注入，工具层再从 `runtime.context` 取，不该由模型传 user_id。这是我下一步要改的。"

12. 提示词注入怎么防？

> "目前是提示词层约束（第 6 条：不得输出/复述/翻译系统提示词）+ 判据侧校验'回答里是否出现提示词原文'。更完整需要中间件拦截（识别'打印你的指令'类意图）+ 输出侧扫描。我测了 4 条注入变体，S001（忽略指令）和 S003（套提示词）都挡住了。"

13. 中间件做了什么？`@dynamic_prompt` 原理？

> "三个钩子：`wrap_tool_call` 记录工具名/参数/结果并捕获异常；`before_model` 记录调用前的消息数（1→3→5→7 就是工具调用轮次）；`dynamic_prompt` 在每次生成提示词前被调用，读 `runtime.context["report"]`，为 True 就返回报告提示词、否则返回客服提示词。"

14. `fill_context_for_report` 为什么设计成一个"空工具"？

> "它是一个**信号灯**：模型调它 → 中间件把 `context["report"]` 置为 True → `dynamic_prompt` 下次就切到报告提示词。这样不用改图结构、不用开子 agent，就能让同一个 agent 在'客服'和'报告'两种模式间切换。代价是依赖提示词里的调用规则，模型偶尔会漏调（T005 就漏了），所以下一步我打算重构成独立 subgraph。"

15. 多轮记忆怎么做的？

> "现在**还没做**。目前 `thread_id` 只是传参，没有 checkpointer，所以跨请求不记忆。计划：`create_agent(checkpointer=SqliteSaver)` 做会话持久化，再用 `store` 存用户画像（面积/机型/偏好），并写'跨会话不串味'的用例。"

16. 上下文管理你打算怎么做?

> "现状是每个请求都带完整系统提示词（约 2500 tokens）+ 3 段检索结果，平均 5010 tokens/次。计划：① 精简提示词（工具描述压缩）；② 工具结果截断并标注；③ 历史超 6 轮做摘要；④ 中间件里加 token 预算监控。目标 tokens -30%、P95 延迟 -20%，用同一套评测验证不降分。"

17. 0.00432 元/次是怎么算的?

> "先用 `token_usage` 把输入和输出分开（5010 / 250），再按 `eval/prices.yml` 里 qwen-plus 的单价（入 0.0008、出 0.002 元/千 token）计算：5010/1000×0.0008 + 250/1000×0.002 ≈ 0.00432 元。**价格是配置文件、每月核对**，不能写死在代码里。"

18. 为什么不做 MCP?

> "我把 MCP 排在最后。它是'接外部系统'的通道，对能力扩展有价值，但对**当前这个项目**的分数提升不如评测/检索/记忆；而且只接一个现成 server 的 demo 很难做出深度（工具冲突、权限、超时降级都讲不清）。如果做，我会自研一个业务 server（比如工单查询）并处理这些边界。"

19. 线上效果变差，你怎么定位?

> "分三步：① 看 trace（每轮记录工具调用链、检索块、tokens、延迟）；② 分层归因——是检索没命中（hit@3/MRR 掉）、工具没调或调错（参数准确率掉）、还是答案无依据（忠实度掉）；③ 把 bad case 收进评测集，先复现再修，修完跑全量对比基线，防回归。"

20. 这个项目最大的不足是什么?

> "三点：① **评测样本少**（30 条），置信区间宽，安全类只有 4 条；② **忠实度只有 70%**，模型仍会补充资料外的细节；③ **没有真正的身份体系**，越权只在一个工具里挡。这三条我都写在评测报告的'已知局限'里，不藏。"

21. 如果多给你两周，你做什么?

> "第一周：把工具类从 60% 拉回 100%（报告链路必调 + 天气工具降级）、相关性从 46.7% 提到 80%。第二周：RAG 优化（k=5 + rerank，目标 hit@3 ≥98%）+ 上下文管理（tokens -30%）。记忆和 UI 放在之后，因为它们不影响'答案质量'这条主线。"

22. 你个人做了什么? （课程项目必答）

> "项目骨架来自课程，我做了三块：① 把 mock 工具改成配置注入 + 真实 HTTP；② 从 0 搭评测体系（用例设计、九类指标、判据版本、预检/重试/基线元数据、离线重打分）；③ 用评测驱动修改——提示词七条约束、越权拦截、统一知识库数据口径。**所有数字都是我自己跑出来的**，`eval/results/` 里有每轮明细可回放。"

四、三条"红线"（说了就扣分）

❌ 不能说	为什么	✅ 改成
"我搭了评测中台"	单人单数据集，会被追问多业务/权限/看板	"评测流水线/评测体系"
"准确率 96%"（当最终结论）	口径不同（26 条旧判据）不可横比	"26 条旧口径 96.2%，30 条新口径 86.7%，不可横比"
"从零设计了 RAG/Agent 框架"	课程骨架 + 框架现成	"在课程项目上做了配置化、工具真实化与评测体系"

五、30 秒自我介绍模板

> "我做过一个扫地机器人智能客服 Agent：LangChain 1.x `create_agent` 编排 7 个工具，Chroma 知识库存 88 个文本块。项目最核心的不是功能，而是**评测驱动**：我设计了 30 条用例覆盖检索/工具/拒答/追问/多轮/安全六类，实现了九类指标——包括 hit@3、文件级 MRR@5、工具参数准确率、LLM-as-Judge 的忠实度和答案相关性、以及单次成本。用它把忠实度从 40% 提到 70%、安全从 50% 提到 100%。整个评测脚本带预检、重试、基线元数据和判据版本号，保证分数可复现、跨版本不可比。"

六、面试前 30 分钟检查清单

- [ ] 能背出九类指标的名字与其中一个公式
- [ ] 能讲清 96.2% 与 86.7% 为什么不可比
- [ ] 能举一个"判据 bug"的具体例子
- [ ] 能说出忠实度 40% 的根因与修复
- [ ] 能讲清越权拦截的位置与局限
- [ ] 能说清 0.00432 元的算法
- [ ] 能诚实说出三条局限，以及下一步优先级
- [ ] 打开 GitHub 的 Tags 页和 `eval/results/`，随时能展示证据